101. Встроенные функции

Использование функций имеет много преимуществ, в том числе:

- Код, находящийся внутри функции, может быть повторно использован.
- Гораздо проще изменить или обновить код в функции (что делается один раз), нежели искать и изменять все части кода в функции main() «на месте». Дублирование кода — хороший рецепт для ошибок и ухудшения производительности.
- Упрощение чтения и понимания кода, так как вам не нужно знать реализацию функции, чтобы её использовать (предполагается наличие информативного названия функции и **комментариев**).
- В функциях поддерживается проверка типов данных для гарантии того, что передаваемые аргументы соответствуют параметрам функции.
- Функции упрощают **отладку вашей программы**.

Однако, одним из главных недостатков использования функций является то, что при каждом их вызове происходит расход ресурсов, что влияет на производительность программы. Это связано с тем, что ЦП должен хранить адрес текущей команды (инструкции или стейтмента), которую он выполняет (чтобы знать, куда нужно будет вернуться позже), вместе с другими данными. Затем точка выполнения перемещается в другое место программы. Дальше все параметры функции должны быть созданы и им должны быть присвоены значения. И только потом, после выполнения функции, точка выполнения возвращается обратно. Код, написанный «на месте», выполняется значительно быстрее.

Для функций, которые являются большими и/или выполняют сложные задачи, расходы на вызов обычно незначительны по сравнению с количеством времени, которое отводится на выполнение кода этой функции. Однако для небольших, часто используемых функций, время, необходимое для выполнения вызова, часто превышает время, необходимое для фактического выполнения кода этой функции. А это, в свою очередь, может привести к существенному снижению производительности.

Язык C++ предлагает возможность совместить все преимущества функций вместе с высокой производительностью кода, написанного «на месте». Речь идет о встроенных функциях. **Ключевое слово inline** используется для запроса, чтобы компилятор рассматривал вашу функцию как встроенную. При компиляции вашего кода, все **встроенные функции** (англ. *«inline functions»*) раскрываются «на месте», то есть вызов функции заменяется копией содержимого самой функции, и ресурсы, которые могли бы быть потрачены на вызов этой функции, сохраняются! Минусом является лишь увеличение компилируемого кода за счет того, что встроенная функция раскрывается в коде при каждом вызове (особенно если она длинная и/или её вызывают много раз). Рассмотрим следующий фрагмент кода:

```
#include <iostream>

int max(int a, int b)
{
    return a < b ? b : a;
}

int main()
{
    std::cout << max(7, 8) << '\n';
    std::cout << max(5, 4) << '\n';
    return 0;
}
```

Эта программа дважды вызывает функцию max(), т.е. дважды расходуются ресурсы на вызов функции. Поскольку max() является довольно таки короткой функцией, то это идеальный вариант для её конвертации во встроенную функцию:

```
inline int max(int a, int b)
{
    return a < b ? b : a;
}
```

Теперь, при компиляции функции main(), ЦП будет читать код следующим образом:

```
int main()
{
    std::cout << (7 < 8 ? 8 : 7) << '\n';
    std::cout << (5 < 4 ? 4 : 5) << '\n';
    return 0;
}
```

Такой код выполнится быстрее ценой несколько увеличенного объема.

Из-за возможности подобного «раздувания», встроенные функции лучше всего использовать только для коротких функций (не более нескольких строк), которые обычно вызываются внутри циклов и не имеют **ветвлений**. Также, обратите внимание, ключевое слово inline является лишь рекомендацией — компилятор может игнорировать ваш запрос на встроенную функцию. Подобное произойдет, если вы попытаетесь сделать встроенной длинную функцию!

Наконец, современные компиляторы автоматически конвертируют соответствующие функции во встроенные — этот процесс автоматизирован настолько, что даже лучше ручной выборочной конвертации, проведенной программистом. Даже если вы не пометите функцию как встроенную, компилятор автоматически выполнит её как таковую, если посчитает, что это способствует улучшению производительности. Таким образом, в большинстве случаев нет особой необходимости использовать ключевое слово inline. Компилятор всё сделает сам.

Правило: Если вы используете современный компилятор, то нет необходимости использовать ключевое слово inline.

Встроенные функции освобождаются от «правила одного определения»

На предыдущих уроках мы не раз говорили, что вы не должны определять функции в **заголовочных файлах**, так как если вы подключаете один заголовок с определением функции в несколько файлов .cpp, то определение функции также будет скопировано несколько раз. Затем, на этапе линкинга, линкер выдаст ошибку, что вы определяете одну и ту же функцию больше одного раза.

Однако встроенные функции освобождаются от этого правила, так как дублирования в исходном коде не происходит — определение функции одно, и никакого конфликта при соединении линкером файлов .cpp возникнуть не должно.

Сейчас это всё может показаться неинтересными пустяками, но чуть позже, когда мы будем рассматривать новый тип функций (дружественные функции), эти *пустяки* пригодятся. И помните, что даже с использованием встроенных функций, вы НЕ должны определять глобальные функции в заголовочных файлах.